

Sieci Komputerowe



⌘ Materiały do Laboratoriów

⌘ Część 4:

⌘ NAT, IPv6

Translacja adresów IP - NAT



- ⌘ Zastosowanie urządzeń konwertujących adresy IP (*Network Address Translation* - NAT) dla sieci lub hostów
- ⌘ Nazwa potoczna, związana z dostarczaną przez NAT funkcjonalnością konwersji: masquerade - translacja adresów pozwala między innymi na „ukrycie” sieci wyizolowanej
- ⌘ Adresy hostów w podsieci, z której ruch wychodzący podlega konwersji są konfigurowane z użyciem tradycyjnego ustawienia domyślnej bramki – będącego jednocześnie adresem hosta IP dokonującego translacji (ruter sprzętowy, serwer itp.).
- ⌘ Bramka taka odbiera datagram i zmienia jego nagłówek IP wpisując inny adres jako nadawcy (zmieniając też często numer portu TCP)
- ⌘ Wadą są trudności z funkcjonowaniem niektórych usług warstw wyższych – użytkujących połączenia ustanawiane w przeciwnym kierunku niż wyjściowy z sieci lokalnej (serwer->klient TCP itp.)

Translacja adresów IP - bloki adresów



- ⌘ NAT może dokonywać translacji w trybie:
 - ☒ Jeden adres IP <-> jeden adres
 - ☒ Wiele adresów IP (pula IP) <-> jeden adres
 - ☒ Wiele adresów IP <-> wiele adresów IP
- ⌘ Konwersja nie musi być wiązania z przejściem datagramu przez bramkę – możliwe jest użycie tzw. rutera na patyku (router on a stick), gdzie datagram po konwersji wraca do tej samej sieci
- ⌘ Z reguły NAT operuje na trzech blokach adresów, przypisanych do tzw. sieci prywatnych. Znaczna większość routerów nie posiada jednak ograniczenia NAD do adresacji sieci prywatnej.
- ⌘ Bloki adresów traktowanych jako przynależne do sieci prywatnych:
 - ☒ Klasa A : 10.0.0.0 – 10.255.255.255
 - ☒ Klasa B : 172.16.0.0 – 172.31.255.255
 - ☒ Klasa C : 192.168.0.0 – 192.168.255.255

NAT - overloading

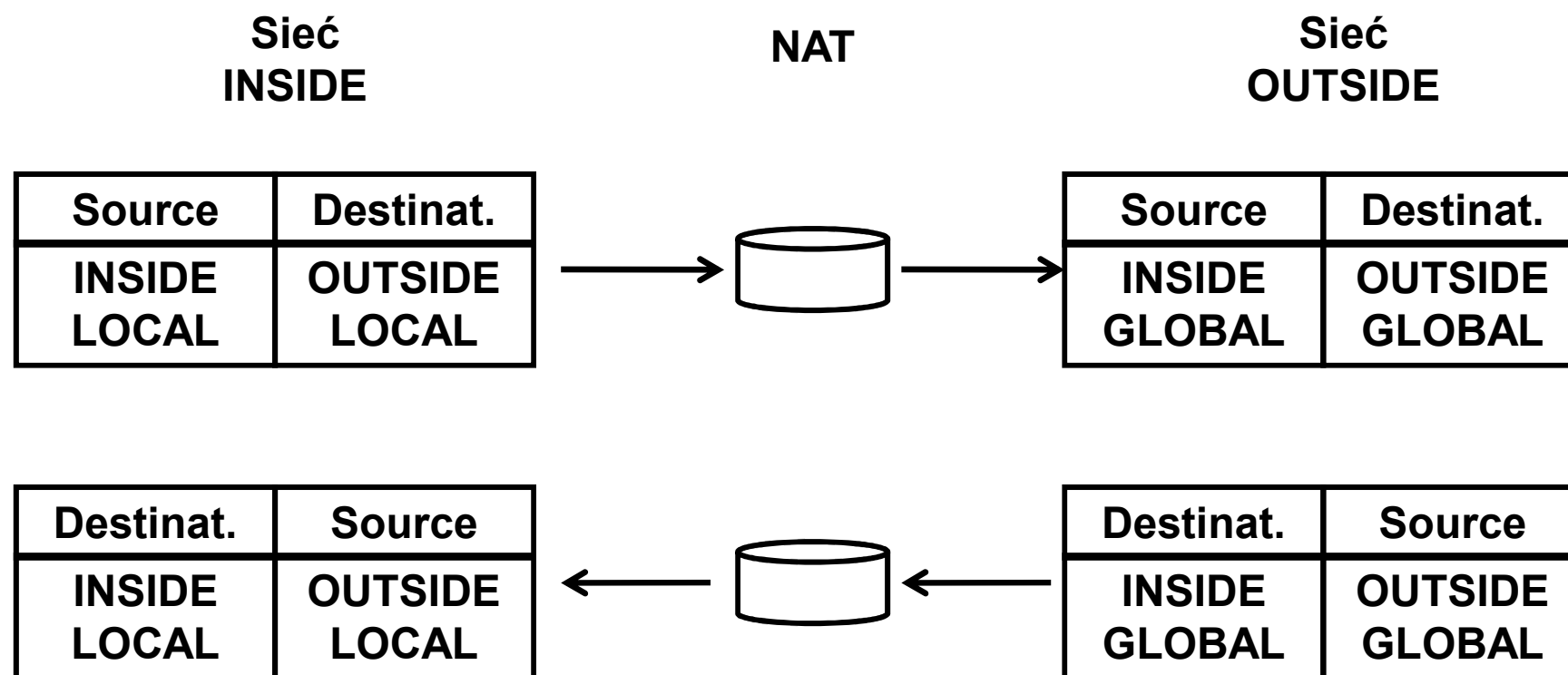


- ⌘ *Overloading* umożliwia stworzenie relacji typu „jeden do wielu” pomiędzy adresami IP podczas translacji (NAT). NAT bez trybu *overloading* pozwala tylko na translację kolejnych adresów IP na kolejne inne.
- ⌘ W większości ruterów funkcjonalność NAT jest konfigurowana w oparciu o tzw. pulę adresów IP (w ten sposób określa się zakres adresów).
- ⌘ Kontrola procesu translacji jest regulowana poprzez standardowe listy kontrolne (ACL)
- ⌘ Wyróżniane są dodatkowo interfejsy, pomiędzy którymi następuje translacja (*inside* i *outside*)
- ⌘ W przypadku stosowania trybu *overloading* możliwe także jest zdefiniowanie puli adresów – lecz wtedy jedno-adresowej i przypisanie jej do interfejsu *inside* (z tego interfejsu odbierane są datagramy od różnych nadawców – więc z różnymi adresami IP do konwersji na „adres nadawcy” w *outside*)

NAT – rodzaje adresów

- ⌘ Podział ze względu na występowanie (uwaga – nie intuicyjny):
 - ⏏ Adres local – występujący w datagramach w sieci inside (prywatnej)
 - ⏏ Adres global - występujący w datagramach IP w sieci outside (publicznej)
- ⌘ Z zachowaniem tego podziału a NAT występują 4 rodzaje adresów:
 - ⏏ Inside local – adres hosta w sieci prywatnej (inside)
 - ⏏ Inside global – adres interfejsu IP w sieci prywatnej, który jest nadawcą datagramów do sieci publicznej po konwersji (występuje w imieniu hostów z sieci prywatnej). Gdy wybieramy opcję overloading adres ten będzie adresem interfejsu outside w routerze NAT. W przeciwnym wypadku – interfejsu inside w tym routerze.
 - ⏏ Outside global – rzeczywisty adres hosta w sieci publicznej (outside)
 - ⏏ Outside local – adres hosta z sieci publicznej, pod którym występuje on w sieci prywatnej (przeważnie taki sam, jak outside global)

NAT – rodzaje adresów



Cisco IOS - konfigurowanie NAT bez puli adresów

⌘ Konfigurowanie interfejsów:

Router(config)#ip routing

Router(config)# int fa 0/0

Router(config-if)#ip address 10.0.0.1 255.255.255.0

Router(config-if)#exit

Router(config)# int fa 0/1

Router(config-if)#ip address 200.200.200.1 255.255.255.0

⌘ Definiowanie stron dla NAT:

Router(config)#int fa 0/0

Router(config-if)#ip nat inside

Router(config)#int fa 0/1

Router(config-if)#ip nat outside

Cisco IOS - konfigurowanie NAT bez puli adresów

- ⌘ Definiowanie list kontrolnych - określają adresy hostów, których datagramy mogą być poddawane konwersji:

Router(config)# access-list 5 permit 10.0.0.0 0.0.0.255

- ⌘ Przypisanie listy do NAT (po stronie inside, wtedy ta strona będzie mogła inicjować translację i połączenia):

Router(config)# ip nat inside source list 5 interface fa 0/1 overload



Sprawdzenie:

Ruter#show ip nat translations

- ⌘ Uwaga: w przypadku istnienia innych ruterów w sieci publicznej, każdy z nich musi posiadać w tablicy rutowania drogę do publicznych adresów rutera NAT (tu: inside public = 200.200.200.2) kierującą do interfejsu outside (tu: 200.200.201.1)

Cisco IOS - konfigurowanie NAT opartego o pulę adresów

⌘ Uwaga – pula może dotyczyć innych adresów niż adres interfejsu routera.

⌘ Konfigurowanie interfejsów:

Router(config)#ip routing

Router(config)# int fa 0/0

Router(config-if)#ip address 10.0.0.1 255.255.255.0

Router(config-if)#exit

Router(config)# int fa 0/1

Router(config-if)#ip address 200.200.200.1 255.255.255.0

⌘ Definiowanie stron dla NAT:

Router(config)#int fa 0/0

Router(config-if)#ip nat inside

Router(config)#int fa 0/1

Router(config-if)#ip nat outside

Cisco IOS - konfigurowanie NAT opartego o pulę adresów

- ⌘ Definiowanie puli adresów dla NAT:

```
Router(config)# ip nat pool pula 200.200.200.1  
200.200.200.254 netmask 255.255.255.0
```

- ⌘ Listy kontrolne określające adresy hostów, których datagramy mogą być poddawane konwersji – wraz z przypisaniem do NAT (po stronie inside, ta strona może inicjować translację i połączenia):

```
Router(config)# access-list 5 permit 10.0.0.0 0.0.0.255  
Router(config)# ip nat inside source list 5 pool pula overload
```

- ⌘ Weryfikacja:

```
Ruter#show ip nat translations
```

Diagnostyka NAT

– Cisco IOS



⌘ Z użyciem puli adresów:

⌘ *Router#show ip nat translations*

⌘	Pro	Inside global	Inside local	Outside local	Outside global
⌘	icmp	200.200.200.2:17	10.0.0.2:17	200.200.200.100:17	200.200.200.100:17
⌘	icmp	200.200.200.3:18	10.0.0.2:18	200.200.200.100:18	200.200.200.100:18
⌘	icmp	200.200.200.4:8	10.0.0.3:48	200.200.200.100:48	200.200.200.100:48
⌘	icmp	200.200.200.5:9	10.0.0.3:39	200.200.200.100:39	200.200.200.100:39

⌘ Bez puli adresów:

⌘ *Router#show ip nat translations*

⌘	Pro	Inside global	Inside local	Outside local	Outside global
⌘	icmp	10.0.0.1:62	10.0.0.2:62	200.200.200.100:62	200.200.200.100:62
⌘	icmp	10.0.0.1:63	10.0.0.2:63	200.200.200.100:63	200.200.200.100:63
⌘	icmp	10.0.0.1:31	10.0.0.3:31	200.200.200.100:31	200.200.200.100:31
⌘	icmp	10.0.0.1:32	10.0.0.3:32	200.200.200.100:32	200.200.200.100:32

NAT a rutowanie IP

⌘ Generalne zasada – konfiguracja NAT **nie decyduje** o tym co zrobić z datagramem IP (np. umieścić w danym interfejsie wyjściowym), lecz jedynie podmienia wartości adresów w datagramie. Więc decyzje o postępowaniu z datagramem są ustalane na podstawie zawartości tablicy rutowania IP (gdy jest to jeszcze istotne).

⌘ Kolejność wykonywania operacji:

Router (config)#ip nat inside

- dokonanie konwersji w datagramie wchodzącym do rutera (po przeprowadzeniu konwersji podejmowane są kroki związane z rutowaniem IP)

Router (config)#ip nat outside

- dokonanie konwersji w datagramie wychodzącym z rutera

NAT - Tworzenie DMZ i mapowań statycznych

- ⌘ Umożliwienie mapowania portów dla strumieni TCP z sieci outside do usług w sieci inside:

*Router(config)# ip nat inside source static tcp 10.0.0.2 8080
10.0.0.1 80*

gdzie *10.0.0.2* to adres inside local, *10.0.0.1* to adres inside global,

- ⌘ Powyższa instrukcja funkcjonuje najczęściej w połączeniu z mapowaniem statycznym
- ⌘ Mapowanie (translacja) statyczna występuje w tablicy NAT cały czas. Zdefiniowanie translacji statycznej:
Router(config)# ip nat inside source static 10.0.0.3 200.200.200.2

Adresowanie IPv6

- ⌘ IPv6 – posiada adresy 128 bitowe
- ⌘ Maska w notacjach IPv6 zapisywana jest na końcu po znaku '/'
- ⌘ Dozwolona jest pełna notacja szesnastkowa z dwukropkami co 16 bitów, oraz wersje ze skracaniem zer:

☒ 6A2E:8BA1:FFFF:FFFF:0:125A:3E:290

☒ 6A2E:0:0:0:0:0:B6:A25E = 6A2E::B6:A25E

☒ 6A2E:A25E:0:0:0:0:0:0 = 6A2E:A25E::

Adres IPv6 Unicast i jego wykorzystanie

⌘ Adres IPv6 składa się z następujących części:

- ☒ 3 bity Format (prefix)
- ☒ 13 bitów TLA (*Top Level Aggregation*) ID – określają identyfikator dostawcy pierwszego poziomu (*Top Level Aggregator*)
- ☒ 8 bitów - zarezerwowane
- ☒ 24 bity NLA (*Next Level Aggregation*) ID - identyfikator dostawcy drugiego poziomu (*Next Level Aggregator*)
- ☒ 16 bitów SLA (*Site Level Aggregation*) ID - lokalny identyfikator sieci (*Site Level Aggregator*)
- ☒ 64 bity - identyfikator interfejsu

⌘ Teoretyczna liczba hostów:

340 282 366 920 938 463 463 374 607 431 768 211 456

Adres IPv6

- unicast

⌘ Komponenty adresu IPv6:

- ☒ TLA odpowiadają wpisom w globalnej tablicy rutowania, może ich być 8192, ilość tę można w przyszłości powiększyć biorąc bity z 8 zarezerwowanych
- ☒ NLA powinny określać cel w ramach jednego TLA, typowo jeden identyfikator może być przyznawany jednej instytucji
- ☒ SLA - pozwala na określenie podsieci lokalnych, może ich być 65535
- ☒ Identyfikator interfejsu jest kojarzony z adresem Ethernet (MAC), ale przeznaczono na niego 64, a nie 48 bitów. Często konstruuje się go z MAC wstawiając po 3-cim bajcie MAC dodatkowe 16 bitów o wartości 0xFFFE.

IPv6 unicast i EUI-64

- ⌘ Notacja EUI-64 (*64-Bit Extended Unique Identifier*) – stosowana gdy w adresie IPv6 istnieje *interface id* (64-bitowa najmłodsza część adresu IPv6) zawierająca *MAC address* (48 bitów) uzupełniony pośrodku wartością 0xFFFE. Przykład definicji adresu dla interfejsu urządzenia (Cisco):

5555:1111:1111:1111::/64 eui-64

Interfejs IPv6 otrzyma wtedy nowy adres będący połączeniem powyższej konstrukcji i MAC (dodatkowo 7-my najstarszy bit MAC jest ustawiany na wartość 1)

- ⌘ Wartości Unicast zarezerwowane w IPv6:
 - 2000::/3 - Global Unicast
 - fc00::/7 - Unique Local Unicast
 - fe80::/10 - Link Local Unicast

IPv6 - adresy specjalne i prefiksy



- ⌘ Przeznaczenie adresu IP określa jego prefiks (określany jako pewna ilość najstarszych bitów przy odpowiedniej masce), np.:
- ⌘ `::/128` lub `0:0:0:0:0:0:0:0` – adres nie wyspecyfikowany (odpowiednik `0.0.0.0/32` w IPv4) – używany do określania dozwolonych klientów dla połączeń
- ⌘ `::/0` `0.0.0.0/0` (odpowiednik `0.0.0.0/0` w IPv4) - używany do określania *default routes* przy rutowaniu
- ⌘ `0:0:0:0:0:0:0:1` - odpowiednik `127.0.0.1` w IPv4
- ⌘ `fc80:prefix::/10` – *unique local address*, służący do (unikatowego także w przypadku wydostania się poza bieżącą sieć) adresowania hostów lokalnych. Nie rutowalny. Często uzupełniony przez EUI-64. *Prefix* to 41+16 bitów (*subnet+link ID*)
- ⌘ `fe80::/10` - *link-local prefix* (odpowiednik adresu auto-konfiguracji w `169.254.0.0/16` w IPv4), każdy interfejs go posiada (awaryjnie). Często jest uzupełniany przez EUI-64

IPv6 - adresy specjalne i prefiksy



- ⌘ fec0::/7 – unikatowe adresy lokalne (*site local*), nie przeznaczone do rutowania (lub rutowalne w bardzo ograniczonych sytuacjach, w grupie kilku wyróżnionych węzłach sieci)
- ⌘ 2001::/?, 2002::/? – adresy przeznaczone do tunelowania (różne technologie, przy maskach '?')
- ⌘ ff01::?, ff02::?, ff05::? – adresy multicast przeznaczone do wspierania funkcji specjalnych (cyfra zmienna oznacza liczbę, która identyfikuje kolejną funkcję). Przykładowe funkcje to to ruch związany z procesami rutowania (np. identyfikacja ruterów, protokoły RIPng, OSPF, EIGRP dla IPv6), DHCP, propagowanie nazewnictwa sieci i łącz, DNS, NTP i inne.
- ⌘ ff00::0/8 są zarezerwowane.

Cechy interfejsów IPv6



- ⌘ Interfejs może mieć wiele adresów IPv6, np.: kilka różnych *link-local address*, *global address* i inne.
- ⌘ Adresy mogą być trwałe lub tymczasowe (mogą być użyte tylko dla konkretnego połączenia wychodzącego, identyfikując klienta + usługę na bazie treści adresu IPv6)
- ⌘ Istnieją konfigurowalne „tablice preferencji”, łączące każdy prefiks danego adresu (*routing prefix*) z tzw. *precedence level* (liczbą, będącą priorytetem). W przypadku posiadania (np. chwilowego) danego adresu – możliwe jest wtedy automatyczne podjęcie decyzji, którego adresu użyć np. jako źródłowego dla połączenia wychodzącego.

Lifetime dla adresów IPv6



- ⌘ Każdy adres IPv6 ma czas życia (domyślnie skonfigurowany jako nieskończony)
- ⌘ Ruter konfigurujący adresy zdalnych interfejsów IPv6 może wymieniać ich adresy podając interfejsom dodatkową wartość *lifetime* adresu
- ⌘ Przeterminowany adres IPv6 przechodzi w interfejsie ze stanu *preferred* do *deprecated* (dalej może być używany, lecz bez możliwości nawiązywania nowych połączeń przy jego użyciu)
- ⌘ Po dalszym czasie przechodzi do stanu *invalid*. Wtedy może być już przypisany do innego interfejsu

IPv6 multicast

⌘ Adres IPv6 multicast zawiera:

- ☒ 8-bitowy prefiks o wartości 11111111 (FF)
- ☒ 4-bitowe pole flag (3 bity używane)
- ☒ 4-bitowe pole scope field identyfikujące unikatowość adresu i (częściowo) przynależność do konkretnej usługi oraz przede wszystkim zasięg, w jakim znajdować się mogą odbiorcy datagramu
- ☒ 112-bitowe pole identyfikatora grupy multicast

⌘ Adresy IPv6 multicast są powszechnie użytkowane do wspierania protokołów Internetu zarządzających ruchem nad IPv6

IPv6 multicast address scopes



- ⌘ Istotne wartości pola *scope* decydującego o klasyfikacji ruchu IPv6 multicast (w poniższych formułach 'x' to dowolna liczba z przedziału 1..8):
- ⌘ ff00::/128-ff0f::/128 – zarezerwowane (nie należy używać)
- ⌘ ffx1::/16 - *multicast interface-local* – datagramy nie wychodzą poza localhost (możliwe jest rozgłaszanie w obrębie jednego hosta, z przekazaniem ruchu do wielu aplikacji)
- ⌘ ffx2::/16 - *multicast link-local* – z przeznaczeniem dla lokalnego segmentu sieci, datagramy nie będą rutowane (odpowiednik IPv4: 224.0.0.0/24)
- ⌘ ffx5::/16 – *multicast site-local* - z przeznaczeniem dla lokalnej sieci fizycznej. W tym przypadku dodatkowo (zależnie od specyfiki urządzeń) – ruch może być blokowany w urządzeniach konwertujących medium (np. w Access Point WiFi)
- ⌘ ffxe::/16 - *global scope* – zwykłe i rutowalne grupy IPv6 multicast

IPv6 – rutowanie i maski



- ⌘ W przypadku hostów nie stosuje się przypisać masek do interfejsów. Natywne rozgraniczenie prefiksu sieci następuje w połowie długości adresu (młodsze 64 bity to identyfikator interfejsu)
- ⌘ Inne niż /64 maski definiowane są w ruterach dla sieci bezpośrednio podłączonych. W praktyce – tylko tych, które nie zawierają hostów docelowych
- ⌘ Host posiada tablicę rutowania. Domyślna reguła powinna kierować ruch na bramkę (uwaga – dotyczy ona wszystkich adresów nie link--local)
- ⌘ W zależności od wybranego adresu różne jest zachowanie hosta-nadawcy (inny rodzaj adresu źródłowego w datagramie IP powoduje inną interpretację lokalnej tablicy rutowania IPv6). Na przykład – gdy wyślemy pakiet multicast typu *global scope* (ffxe:) zostanie użyty adres nadawcy *global*. Gdy *local scope* (ffx2::) - adres nadawcy *unicast local scope*.

Cisco IOS – konfigurowanie IPv6



- ⌘ IPv6 posiada własną implementację stosu TCP/IP – w konsekwencji w routerze znajdują się dwie tablice routowania
- ⌘ Osobny zestaw komend do zarządzania IPv6 – dostępny w drzewie komendy ipv6
- ⌘ Protokoły warstwy 4 ISO OSI i wyższych nie są już zależne od tego podziału
- ⌘ Definiowanie adresu IPv6 dla interfejsu:
Router(config-if)#ipv6 address 1:2:1::2/64
- ⌘ Definiowanie adresu IP w standardzie EUI-64 dla interfejsu:
Router(config-if)# ipv6 address 1111:1111:1111:1111::/64 eui-64
- ⌘ Diagnostyka:
Router# show ipv6 int brief
Router# show ipv6 protocols
Router# show ipv6 tunnel

Cisco IOS – IPv6 i tablice rutowania

- ⌘ Przykład definiowania reguły rutowania statycznego:

Router(config)#ipv6 route 1:1::1/64 1:2:1::1

- ⌘ Definiowanie domyślnej reguły rutowania:

Router(config)#ipv6 route ::/0 1:2:1::3

- ⌘ Wtedy zawartość tablicy rutowania:

Router#show ipv6 route

(...)

*C 1:2:1::/64 [0/0]
via ::, FastEthernet0/0*

*S 1:1::/64 [1/0]
via 1:2:1::1*

*S ::/0
via 1:2:1::3*

IPv6 i IPv4

– mapowanie adresów

- ⌘ Użytkowanie IPv6 jest obecnie w dużej mierze oparte na mapowaniach tych adresów wykorzystujących poprzednią wersję protokołu (IPv4). Technika ta określana jest skrótem **6to4**. Mapowanie adresów:

2002:xxxx:xxxx::/16 <-> xx.xx.xx.xx

gdzie xxxxxxxx to treść adresu IPv4 (reprezentację dziesiętną należy zamienić na heksadecymalną przy zapisywaniu tej treści)

- ⌘ Istnieją także adresy IPv6 mapowane z IPv4 w inny sposób. Powstały tu adres jest określany mianem **IPv4-mapped IPv6** lub **IPv4 compatible address**.

0:0:0:0:0:FFFF:x.x.x.x/96 lub 0:0:0:0:0:0:x.x.x.x/96

gdzie x.x.x.x to adres IPv4 (stosuje się reprezentację dziesiętną, która jest automatycznie zamieniana na heksadecymalną). Zapis adresu zawiera więc zarówno symbole „:” jak i „.” i taka jego postać jest przetwarzana przez systemy operacyjne urządzeń w wydawanych im komendach

Tunelowanie IPv6 – mechanizm 6in4



- ⌘ Mechanizm polega umieszczaniu datagramów IPv6 w tunelach punkt-punkt tworzonych w sieciach IPv4 (następuje enkapsulacja w datagramach IPv4)
- ⌘ Do oznaczenia protokołu tunelowania (6in4) w datagramach IP wykorzystywana jest wartość 41
- ⌘ Tunele są zestawiane manualnie przez odpowiednią konfigurację ruterów-bramek
- ⌘ Istnieje możliwość zestawiania tuneli „dynamicznych” ('proto-41 heartbeat' tunnels) – gdzie przeciwny koniec tunelu może migrować pomiędzy kilkoma hostami (ruterami). Nowy adres tunelu jest wtedy przekazywany poprzez komunikat heartbeat.

Cisco IOS – konfigurowanie tunelu 6in4

- ⌘ Uruchomienie routowania IPv6:

Router(config)#ipv6 unicast-routing

- ⌘ Definiowanie adresacji IP dla interfejsu

Router(config)#interface FastEthernet 0/0

Router(config-if)#ip address 200.200.200.2 255.255.255.0

- ⌘ Tworzenie tunelu (*tunnel destination* znajduje się w odległej sieci) :

Router(config)#interface Tunnel 1

Router(config-if)#no ip address

Router(config-if)#ipv6 address 1::1/64

Router(config-if)#tunnel source FastEthernet 0/0

Router(config-if)#tunnel destination 200.200.201.2

Router(config-if)#tunnel mode ipv6ip

Przekazywanie IPv6 nad IPv4 – wykorzystanie 6to4

- ⌘ Bezpośrednie mapowanie adresów (6to4) nie wymaga ręcznego konfigurowania adresów tuneli pomiędzy sieciami IPv6.
- ⌘ System wykorzystujący tunele bazuje na tu stosowaniu indywidualnych hostów lub ruterów brzegowych z adresami tworzonymi zgodnie z 6to4, komunikujących sieci IPv6 (tzw. wyspy IPv6). W obydwu przypadkach wymagane jest posiadanie adresu IPv4 w sieci globalnej. Zgodnie z 6to4 emulowany jest wówczas pseudo-interfejs IPv6 o adresie: 2002:xxxx:xxxx::/16 lub /48 (gdzie xxxx:xxxx to globalny adres IP)
- ⌘ Ruter operujący w adresacji 6to4 automatycznie tuneluje datagramy IPv6, gdy posiadają one prefix 2002:.
- ⌘ Tunele (podobnie jak w przypadku zwykłego tunelowania 6in4) korzystają z protokołu tunelowania opisywanego wartością identyfikatora protokołu: 41, lecz adresy końców tuneli są ustalane na podstawie treści IPv6

Cisco IOS – konfigurowanie 6in4 z wykorzystaniem 6to4

- ⌘ Uruchomienie routowania IPv6:

Router(config)#ipv6 unicast-routing

- ⌘ Definiowanie adresacji IP dla interfejsu

Router(config)#interface FastEthernet 0/0

Router(config-if)#ip address 200.200.200.1 255.255.255.0

- ⌘ Tworzenie tunelu (adres IPv4 *tunnel destination* nie musi być wytypowany – zostanie ustalony na podstawie IPv6, gdy ktoś będzie potrzebował skomunikować się z następną bramką IPv6):

Router(config)#interface Tunnel 1

Router(config-if)#no ip address

Router(config-if)#ipv6 address 2002:C8C8:C801::1/16

Router(config-if)#tunnel source FastEthernet 0/0

Router(config-if)#tunnel mode ipv6ip 6to4

Wartość *C8C8:C801* w adresie IPv6 jest odpowiednikiem adresu IPv4 *200.200.200.1* (w notacji *hexadecymalnej*)

Cisco IOS – IPv6, konfigurowanie RIPng

⌘ Włączenie:

```
Router(config)# ipv6 unicast-routing  
Router(config)# ipv6 router rip ripper
```

⌘ Konfigurowanie interfejsu i jednocześnie przypisanie sieci do RIPng (w konfiguracji interfejsu):

```
Router(config)# interface FastEthernet0/0  
Router(config-if)# ipv6 address  
1113:1112:1112:1112:1112:1112:1112:1115/112  
Router(config-if)# ipv6 rip ripper enable  
Router(config)# interface FastEthernet0/1  
Router(config-if)# ipv6 address  
1112:1112:1112:1112:1112:1112:1112:1114/112  
Router(config-if)# ipv6 rip ripper enable
```

⌘ Diagnostyka:

```
Router# show ipv6 rip
```

Cisco IOS – IPv6, konfigurowanie OSPF i EIGRP

⌘ Włączenie EIGRP:

```
Router(config)# ipv6 router eigrp 1234
```

⌘ Zarejestrowanie sieci:

```
Router(config)#int fa 0/0
```

```
Router(config-if)#ipv6 eigrp 10
```

⌘ Włączenie OSPFv3:

```
Router(config)#ipv6 router ospf 10
```

```
Router(config-router)#router-id 1.1.1.1
```

```
Router(config-if)#ipv6 ospf 10 area 0.0.0.0
```

⌘ Diagnostyka

```
Router#debug ipv6 ospf packet
```

```
Router#debug ipv6 ospf hello
```

```
Router#sh ipv6 ospf neighbor
```

Użytkowanie adresów IPv6 w systemie operacyjnym

- ⌘ W przypadku typowania URL stosujemy nawias [] - należy go wpisać do URL, np.:
 - ⏏ http://[adres IPv6]/
 - ⏏ https://[adres IPv6]:443/
- ⌘ Ścieżka UNC wykorzystująca IPv6 wygląda następująco:
 - ⏏ \\1111-1-1-1-1-1-1-1111.ipv6-literal.net
(w adresie stosowane są kreski zamiast `:`, oraz przyrostek DNS typujący adres IPv6)
- ⌘ W DNS:
 - ⏏ Do konwersji (*name resolving*) służy rekord AAAA, analogicznie do rekordu A w przypadku DNS dla IPv4
- ⌘ Istnieje protokół ICMPv6 – definiujący odpowiedniki komend ICMP dla IPv4

Użytkowanie adresów IPv6 w systemie operacyjnym

⌘ Narzędzia systemowe dla Windows:

⏏ `ping -6 1111:1:1:1:1:1:1:1111`

⏏ `ping -6 ipv6.google.com`

⌘ Konfigurowanie interfejsów i tablicy routowania: narzędzia *netsh* lub *ipv6*, przykłady wywołań:

⌘ `netsh interface ipv6 add address "Local Area Connection 1" 1111::2`

⌘ `netsh interface ipv6 delete address "Local Area Connection 1" 1111::2`

⌘ `netsh interface ipv6 show route`

⌘ `netsh interface ipv6 add route ::/0 "Local Area Connection 1" 1111::1`

⌘ `netsh interface ipv6 delete route ::/0`

NAT w IPv6



- ⌘ NAT, z uwagi na szeroki zakres adresacyjny IPv6, nie jest technologią bezpośrednio z IPv6 powiązaną (powstał nad IPv4 z uwagi na braki w adresacji)
- ⌘ Implementowane jest prowadzenie konwersji host-host (bez techniki *overloading*) – technologia nosi nazwę NAT66. Jest to tak zwany wariant *stateless* (brak *overloading*)
- ⌘ NAT stosowany jest dość często do prowadzenia konwersji IPv4 <-> IPv6 - gdy hosty nie posiadające przeciwległych adresów (odpowiednio IPv4 i IPv6) potrzebują się komunikować. Technologia nosi nazwę NAT64. W jej przypadku konieczna jest translacja wszystkiego, co napływa do interfejsu IPv6 (nie są definiowane pule adresów IP). Po stronie IPv6 technika używa pseudo-interfejsów IPv6 (generuje adresy IPv6 nadawcy). Po stronie IPv4 – używany jest adres IPv4 urządzenia, które dokonuje translacji rutera.

Cisco IOS – IPv6, konfigurowanie NAT64

- ⌘ Konfigurowanie interfejsów (jeden z nich to IPv4, drugi – IPv6) i włączenie NAT64

```
Router(config)# interface FastEthernet0/0
```

```
Router(config-if)# ipv6 address 1111::1112/112
```

```
Router(config-if)#nat64 enable
```

```
Router(config)# interface FastEthernet0/1
```

```
Router(config-if)# ipv6 address 200.200.200.1 255.255.255.0
```

```
Router(config-if)#nat64 enable
```

- ⌘ Włączenie routingu i ustalenie drogi do następnej bramki IPv6:

```
Router(config)# pv6 unicast-routing
```

```
Router(config)#ipv6 route ::/0 1111::1111
```

- ⌘ Zmiana docelowej puli IPv6 dla translacji z 64:FF9B::/96 (IANA) na własną:

```
Router(config)#nat64 prefix stateful 2222::/96
```